

How to build custom scanners for web security research automation

How to build custom scanners for web security research automation - James Kettle, PortSwigger.

- Published: 2023-10-03
- Original: <<https://portswigger.net/research/how-to-build-custom-scanners-for-web-security-research-automation>>
- Preserved from: <https://portswigger.net/research/how-to-build-custom-scanners-for-web-security-research-automation> (live) on 2026-09-16
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

How to build custom scanners for web security research automation | PortSwigger Research

How to build custom scanners for web security research automation

[image: James Kettle]

James Kettle

Director of Research

[@albinowax](#)

-

Published: Tuesday, 3 October 2023 at 13:34 UTC

-

Updated: Thursday, 1 August 2024 at 08:26 UTC

-



In this post, I'll share my approach to developing custom automation to aid research into under-appreciated attack classes and (hopefully) push the boundaries of web security.

As a worked example, I'll build on my research from [Smashing the state machine: the true potential of web race conditions](#). If you haven't already seen it, the [DEFCON recording of this presentation](#) is now available and probably worth a watch.

Identify the opportunity

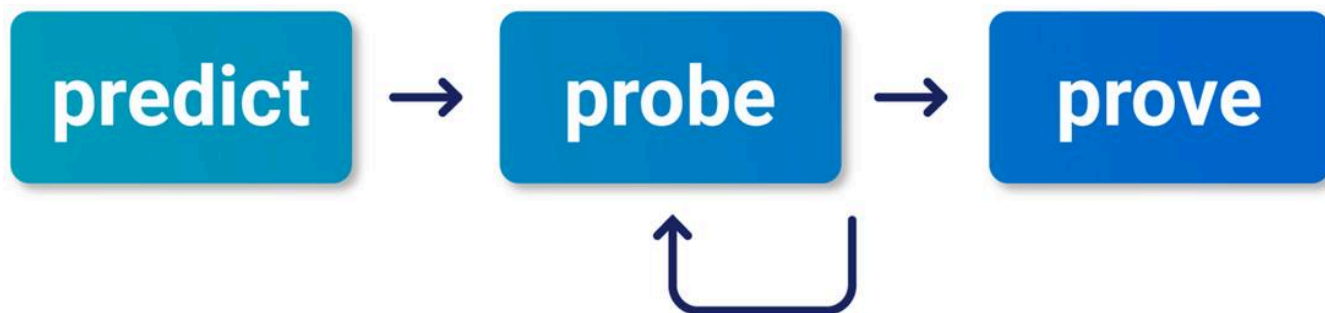
Do you think it's possible to create a scanner that can automatically detect web [race conditions](#)? I initially dismissed the idea, as the race conditions I found required triggering complex, multi-step authenticated interactions with websites and spotting subtle side-effects.

Over the course of this research, I noticed that race conditions often occur in clusters. They bubble up from shaky libraries and frameworks, so if you spot one in a website it's likely that others lurk nearby. This meant that automated detection of race conditions might be valuable even if the detected races were themselves harmless.

I decided that this idea was worth exploring based on my familiarity with the topic, novel tooling in the form of the single-packet attack, and a test-bed that meant I could try out the concept in a day or two.

Avoid over-committing

When attempting to automate something tricky, a common pitfall is to try to automate too much and ultimately fail to achieve anything useful. To avoid this, I like to examine my manual testing methodology and identify the smallest, earliest step that I could plausibly automate. Here's the manual testing process I use for race conditions:



Since the 'predict' phase is just about efficiency, we can skip this and simply try to automate the 'probe' phase. The goal of this phase is to use a batch of concurrent requests to trigger an 'anomaly' and prove that an endpoint might have a sub-state.

Embrace the unexpected

I wrote code to send a request ten times sequentially, then resend it ten times in under 1ms using the single-packet attack. I anticipated that on a race-prone website, the concurrent requests might trigger a 50X error response.

At this point I could have improved efficiency and reduced false positives by only targeting dynamic-looking endpoints, and only reporting responses with 50X codes. However, the best research discoveries often come from unexpected outcomes. This means that it's important to avoid writing your expectations into the code. I deliberately left room for unexpected outcomes by testing all observed requests regardless of what they were for, and reporting any difference in status-code.

When it comes to research I'd rather have false positives than false negatives.

Make iteration easy

This approach inevitably results in a flood of false positives at the start, so it's crucial that making iterative improvements is painless.

I implemented this in a Burp Suite extension, and as a testbed I used a project file containing the homepage and resources from around ~30,000 websites with bug bounty programs. For more details on this setup, check out [Cracking the lens](#).

For a full run, I just select all the requests in the proxy, right click, and launch the scan. It prioritises shorter domains so results on high-profile targets tend to turn up quickly.

Automate your triage

I typically manually triage a small portion of the findings, then analyse my triage process and automate it. While processing the results I found myself:

- Ignoring findings where crucial requests just timed out
- Ignoring findings with 429 status codes as these are just rate-limits
- Ignoring findings with 502/503 as these indicate back-end timeouts
- Trying extra sequential requests after the concurrent batch
- Adding cache-busters to filter out cache behaviour

Implementing this filter process in my scan-check and re-running it left me with a number of promising findings:

- Assorted curious 50X and 307 codes
- A webserver that issues a cryptic '501 Not Implemented' response claiming it doesn't support GET requests when you use the single-packet attack.
- A website that triggers a server-side request to a back-end system for SSO purposes. The single-packet attack overloaded the back-end and triggered an error message disclosing the full URL.
- Another server which intermittently issued a 400 Bad Request response when hit with synchronised requests. This is curious because it suggests that there may be a race condition in their request parsing, which might enable a desync attack.

Unfortunately none of these left me with a clear route forward other than in-depth manual investigation, which I didn't have time for before the conference I was targeting (Nullcon Goa).

Abuse gadgets

What I needed was an approach that would detect behaviour that was obviously dangerous. But what dangerous race-condition can you directly detect from a site's homepage? Well, now and then I've seen reports of applications and caches getting mixed up and either sending responses to the wrong people, or serving up raw memory. The most notorious example of this is, of course, [Cloudbleed](#).

How can we tell if we've received a response intended for someone else? As a human it's easy, and an LLM could probably tell at the price of terrible performance, but it's a tricky question for crude, regex-level automation.

This is where gadgets come in. Gadgets are helpful features present on some websites that make vulnerability detection easier. We can lean on gadgets to quickly and easily explore whether a concept is worth investing more time in. Relying on gadgets for vulnerability detection will cause a lot of false negatives, but during the early stages of research it's worth the trade-off for development speed.

Quite a few websites embed data about the user's request in order to expose it to client-side JavaScript. This typically includes the user's IP address, and request properties like the URL and User-Agent. On sites containing this type of gadget I could detect race-infoleak vulnerabilities by placing a unique 'canary' parameter in every request, then analysing each response to see if it contained a canary from a different request.

This approach initially flagged a lot of websites, but most of them just had cache-poisoning via an [unkeyed query string](#).

After filtering out the cache poisoning and other 'canary storage' behaviour via some more code tweaks, some genuine findings remained. The best example was a certain website where thanks to a race condition, you could obtain the URLs that live users were accessing simply by repeatedly fetching the homepage:

```
window.PAGE_STATE={...{"params":{"utm_source":"bing",...
```

This was perfect for Nullcon; I knocked together a couple of slides and released the scan-checks in [Backslash Powered Scanner](#). You can install it via the BApp store, and [peruse the code on Github](#).

Wrapping up

As we've seen, research-oriented scanning is quite different to building a normal scanner so please be careful when cross-applying this advice to other use-cases.

If you'd like to try your hand at custom automation, the [new BChecks feature](#) in Burp Suite is designed to make this extra accessible.

If you found this useful, you might also enjoy the presentation [Hunting evasive vulnerabilities: finding flaws that others miss](#) where I take a look at research automation from a different angle.

In my next post I'll continue the race condition theme and look beyond HTTP/2 to explore which other protocols support the single-packet attack.

[automation race condition How to research](#)

[Back to all articles](#)

Archived from <https://portswigger.net/research/how-to-build-custom-scanners-for-web-security-research-automation>. Rendered offline from the local Markdown copy.